

# Computer Science & IT

## Compiler Design



Introduction

Lecture No. 1



By- Ravindra Sir



# Recap of Previous Lecture



Topic

Topic

Topic

RBR

III - 86

IV - 13

IISC ME

ME thesis - Compilers  
C-compiler - code optimization

S-grade

S  
A  
B  
C  
D  
E



# Topics to be Covered



Topic

CD-118C-YNS

Topic

Assignment → Design & Comp

Topic

Lance → FE ✓

Bankend - 2000-2500

3 months lines

90%  
CN ✓



fav Subjects

PW

TOL ✓

CD ✓

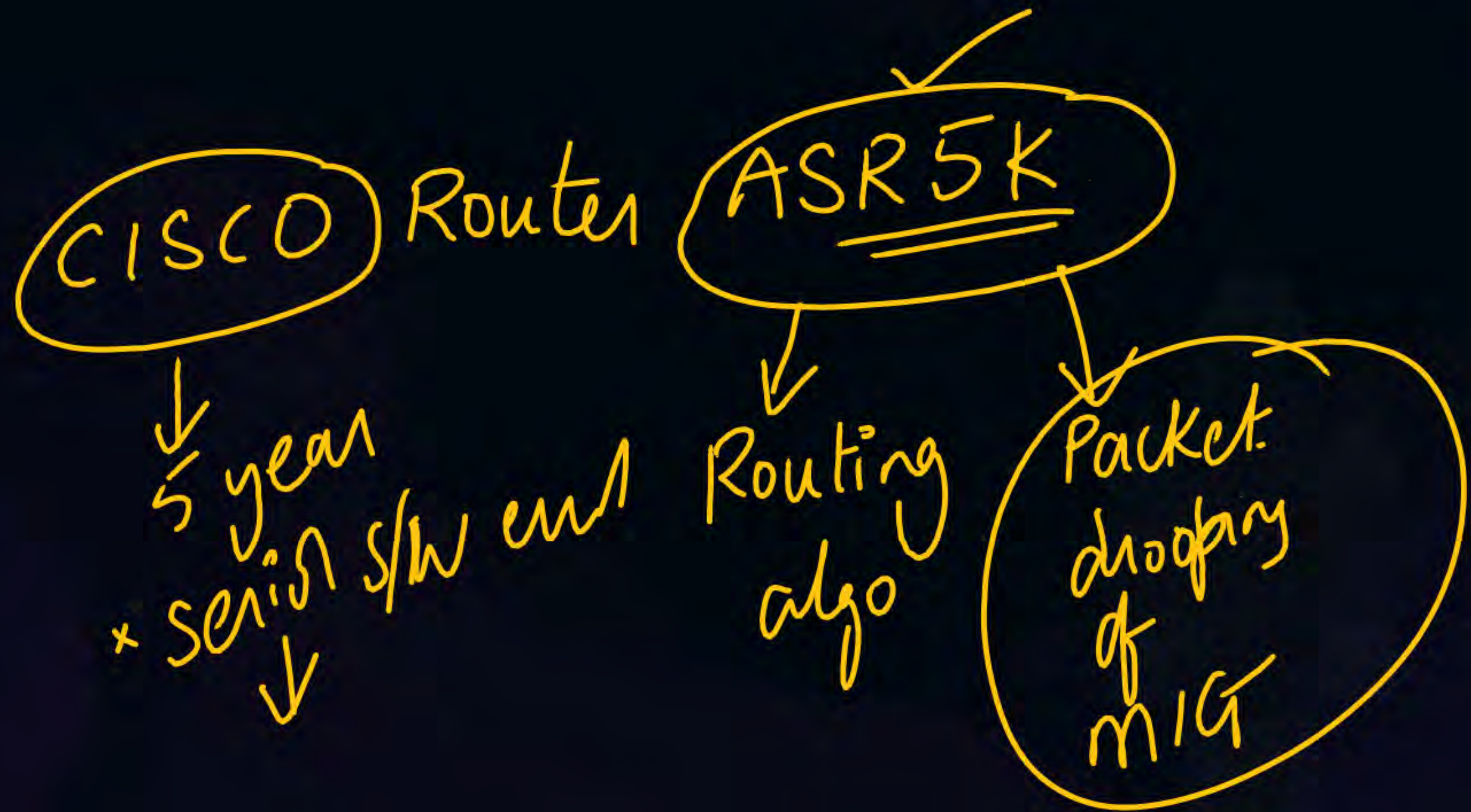
✓ EN ✓  
DBMS ✓

fav

✓  
Textbook ✓

✓  
RFC ✓  
///

CISCO  
↓  
mig





$S \rightarrow (\underline{S}) / \epsilon$

LL(1) parser

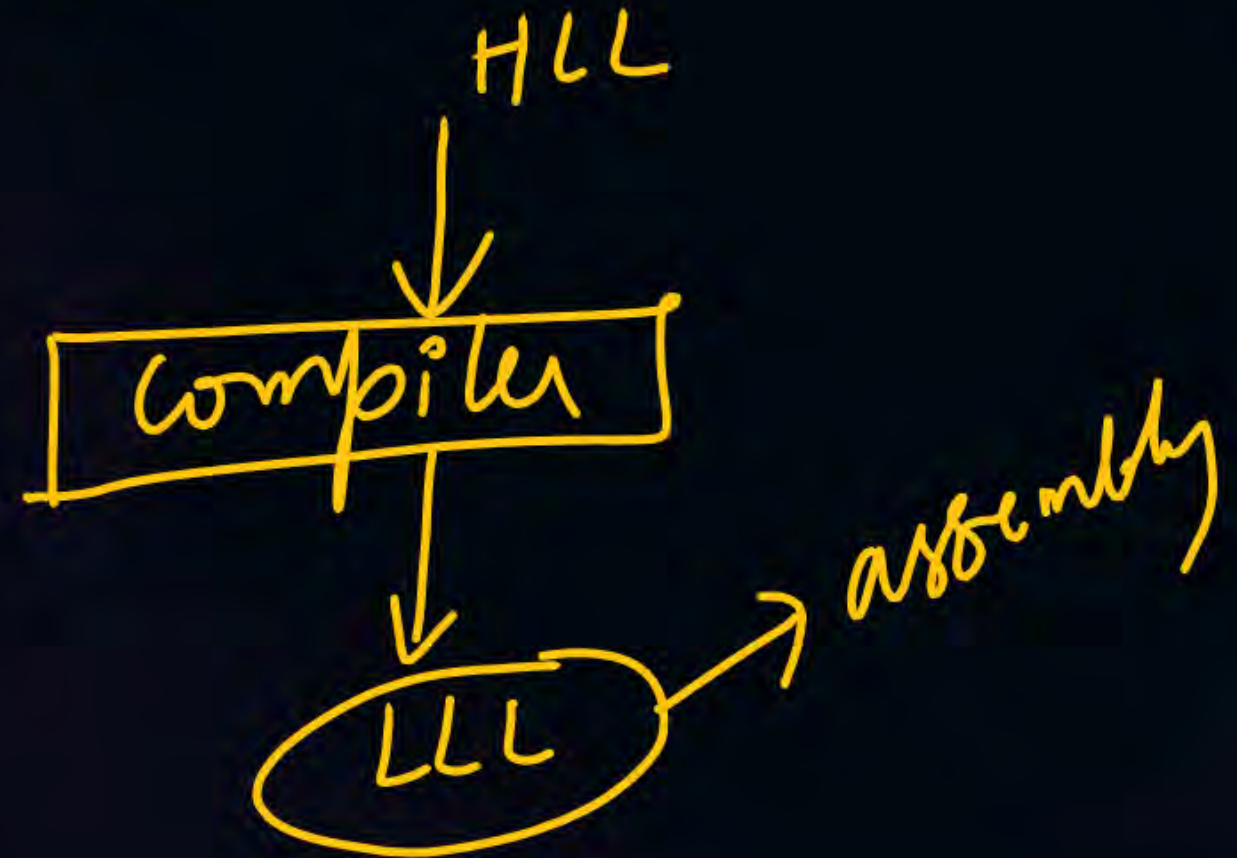
algo

|    |   |    |   |   |    |
|----|---|----|---|---|----|
| \$ | ) | \$ | ) | ) | \$ |
| ↑  |   |    |   |   | ↑  |

# Compiler design



The main aim of compiler is to convert high level language into low level language





→ John Backus at IBM took 18 years  
to build first complete compiler for

Fortran in 1957.

→ Students at IITSc, build C-compiler in 4 months



Building a compiler requires knowledge of





Building a compiler requires knowledge of  
 ✓ → Programming languages (Parameter passing, Scope, memory allocation etc)

\* → Theory (automata, context free languages)

\* → Algorithm and data structures (hash tables, graph algo,

\* → Computer architecture (assembly language) dynamic programming

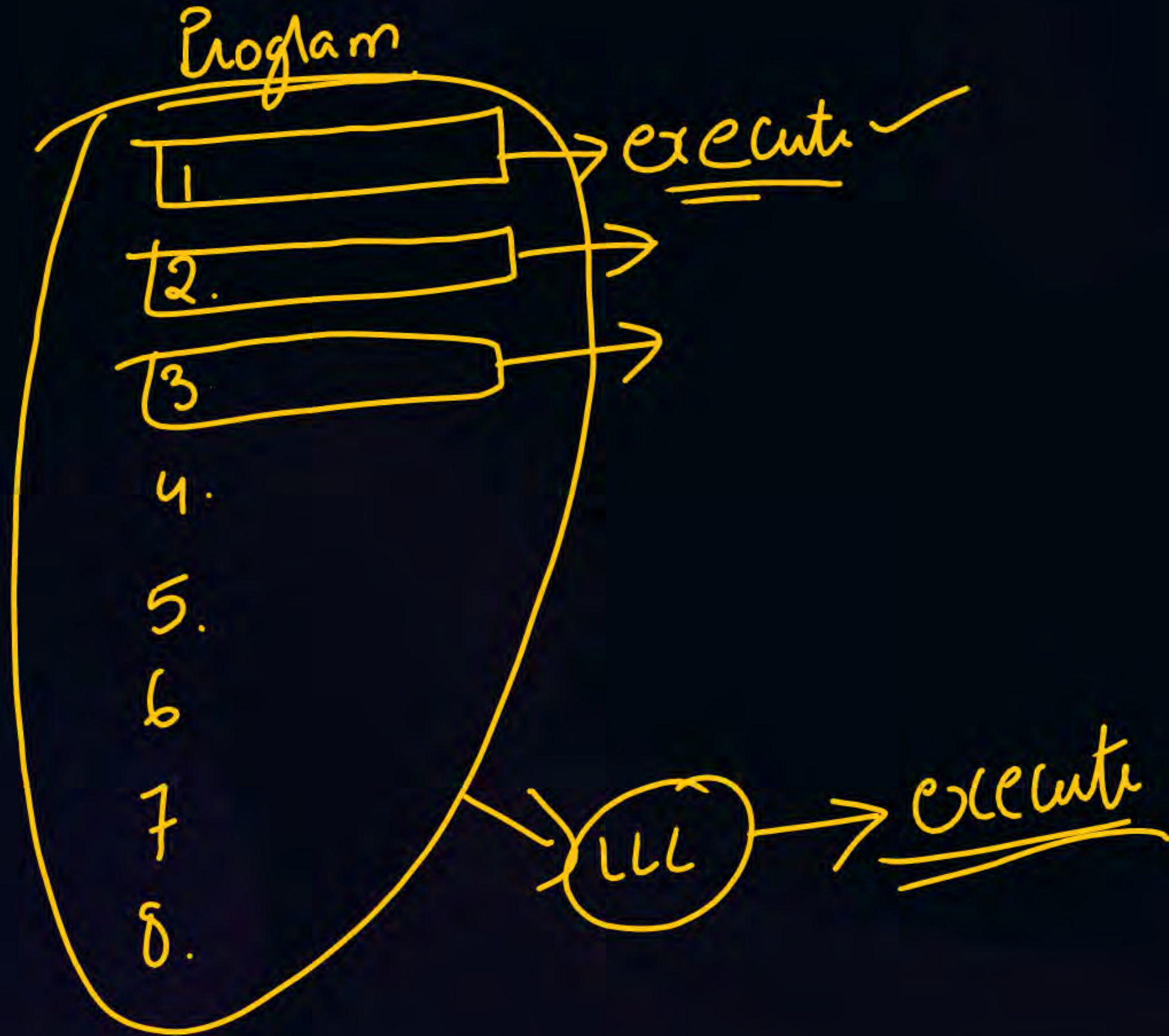
\* → Software engineering ✓



## Interpreter:-

An interpreter works similar to a compiler in most of the phases like lexical, syntax and semantic analyzers. This analysis is performed for each single statement and when the statement is end free, instead of generating the corresponding code, the statement is executed and the result of execution is displayed.









In interpreter every time a program has to be executed, it is processed and checked for errors, it is slower than compiled program.

→ Some languages like Java are both compiled and interpreted. The Java program is first compiled into byte code. This byte code is interpreted.



## Adv of interpreter:-

- If a program is simple and is to be executed only once, an interpreter is preferred.
- Interpreters are more portable than compilers.

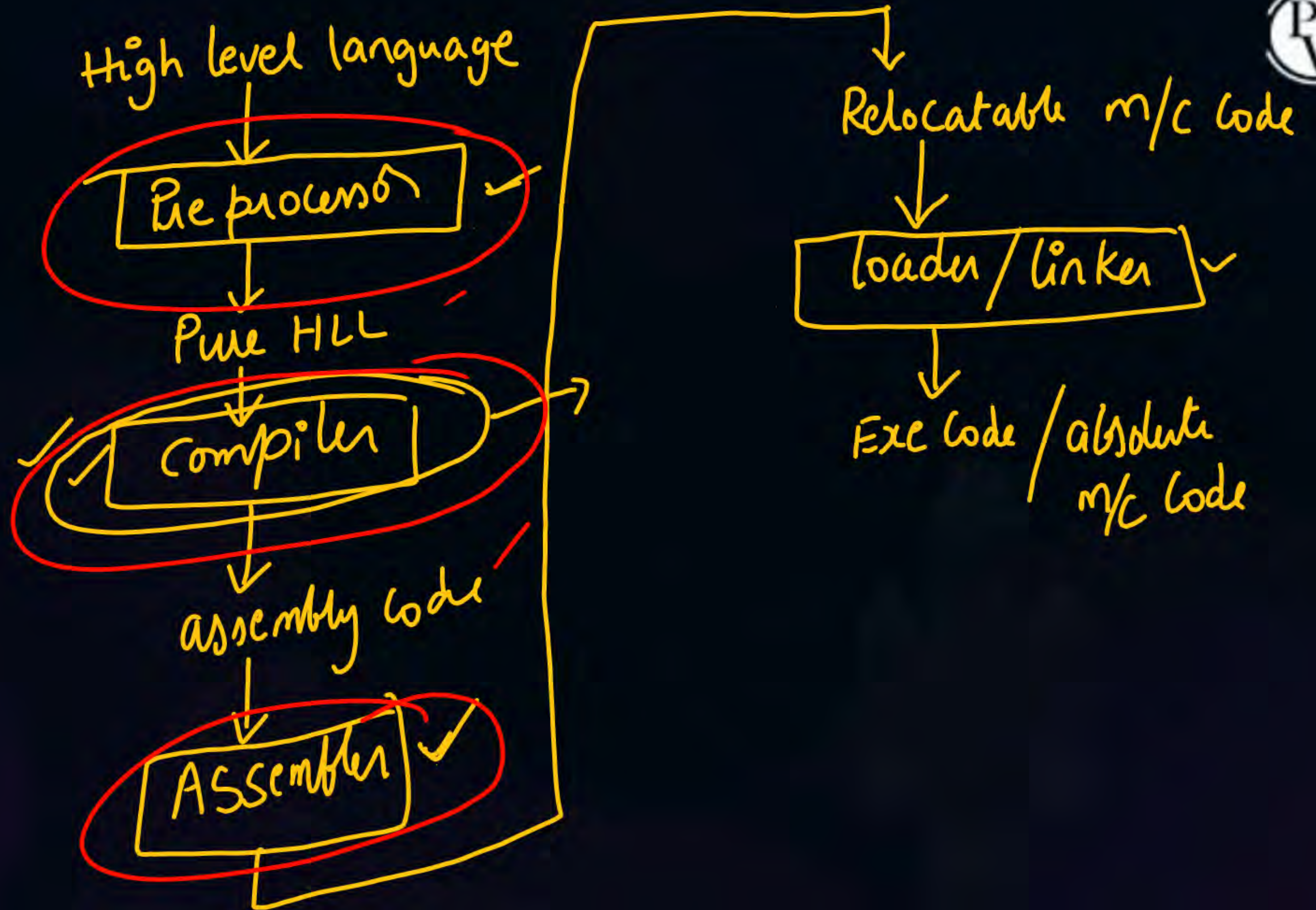




Certain language features are supported by interpreters

Ex:- dynamic types. In SNOBOL, the type of a variable is dependent on the value that is assigned to it any time







## Preprocessor:-

a) macro processing:- It performs macro expansion. All macros are identified and substituted with their respective values.

Ex:- #define PI 3.14

b) File inclusion:- The statements like #include <studio.h> will be replaced by the files.

c) Language extensions:- A preprocessor can add new functionalities to extend the language. Ex:- adding database querying facilities to the language.



#define

#include

Pure HLL ✓



Assembler:-

Assembly code is nemonic version of m/c code in which names rather than the binary values for m/c instructions and memory addresses are used

$$\begin{array}{c} 101101011001011 \\ \hline \text{add} \\ \hline \end{array}$$

$$\begin{array}{c} a \quad b \\ \swarrow \quad \searrow \end{array}$$



An assembler need to assign memory locations & addresses to the the symbols/identifiers.

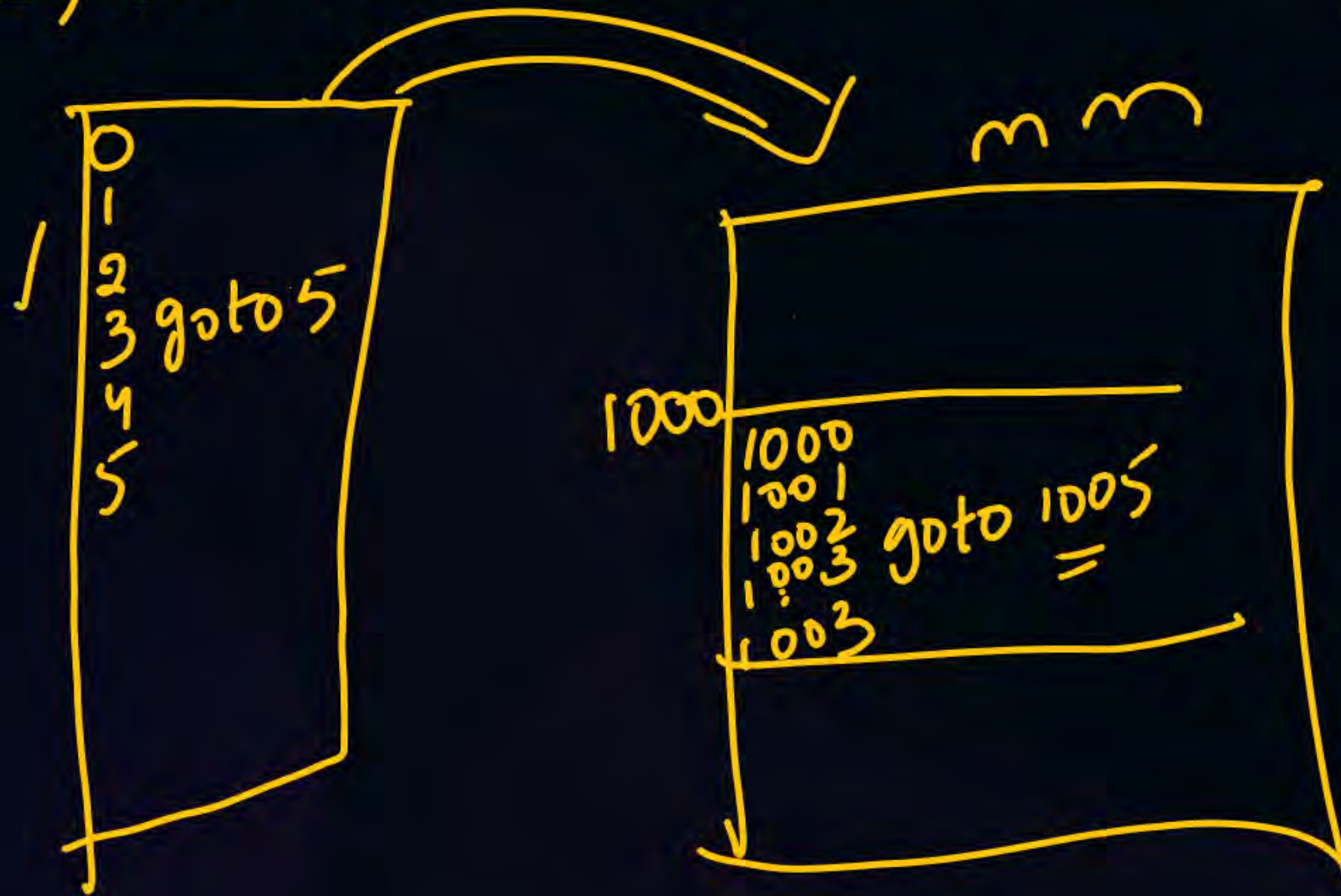
a ✓  
[ ]  
100 ✓





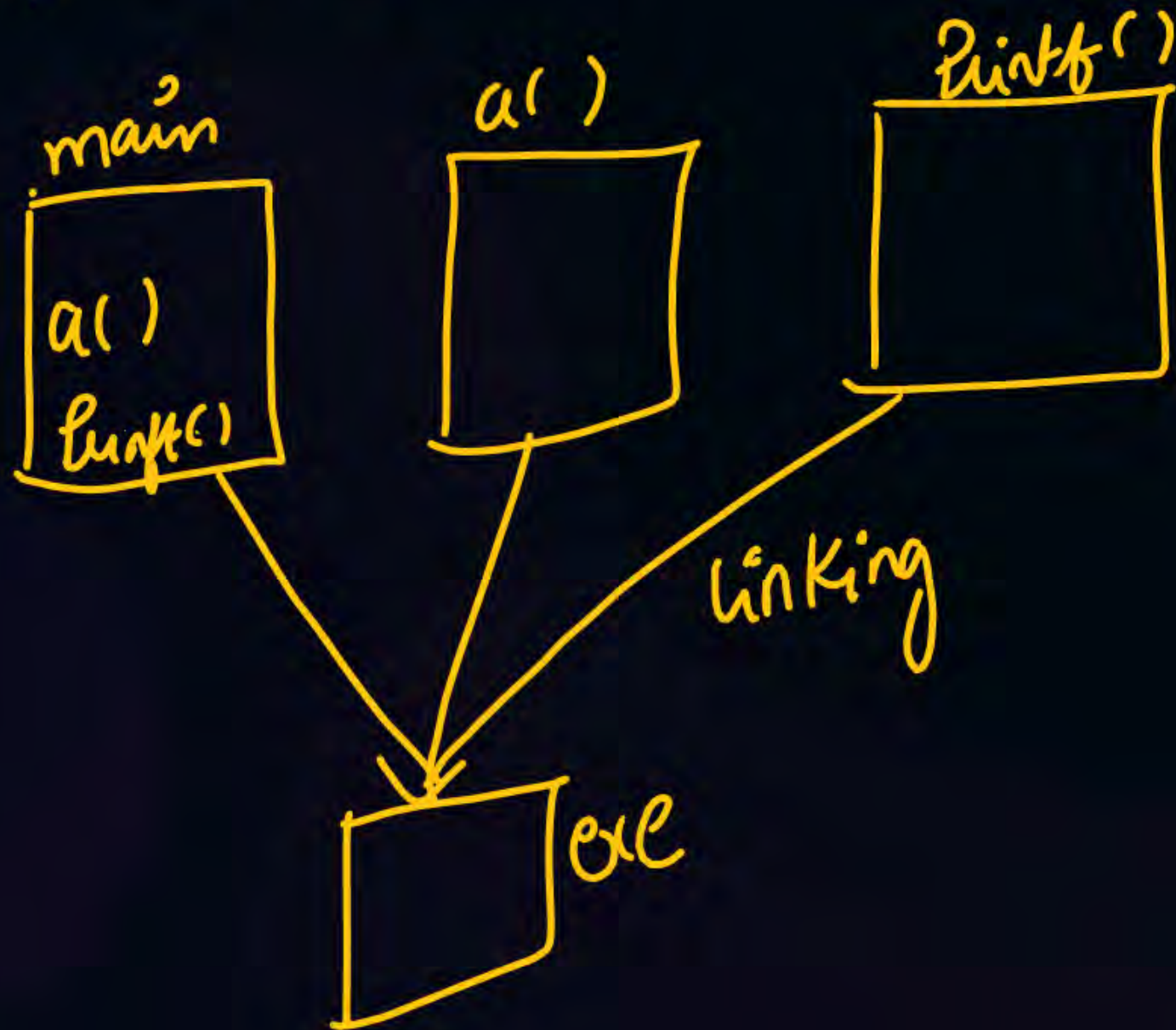
# Loader/Linker :-

To convert the relocatable m/c code to executable code, one or more translators are required





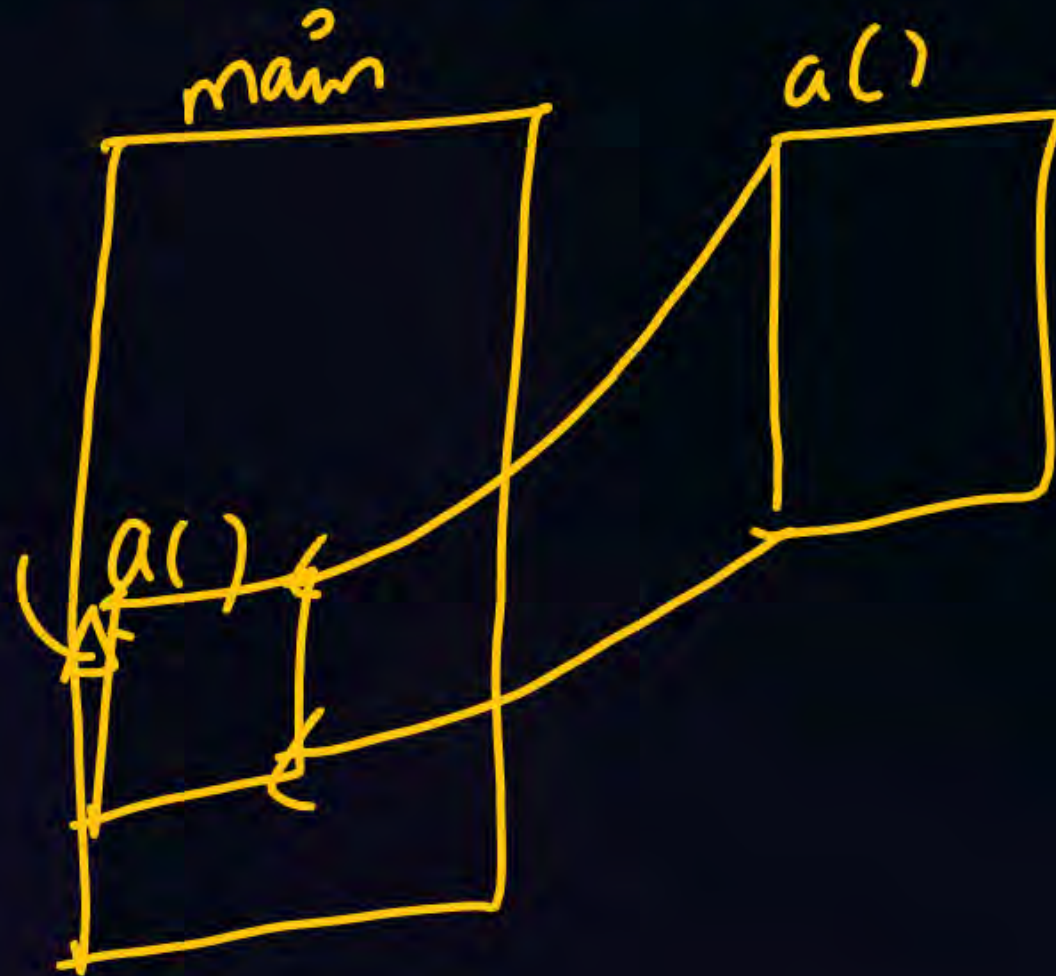
linking:- This is a process where a linker takes several object files and libraries as I/p and produces one file as output





linker:-

it retrieves from the I/p files the code of all procedures that are referenced and resolves all external references to actual machine addresses





linker:-

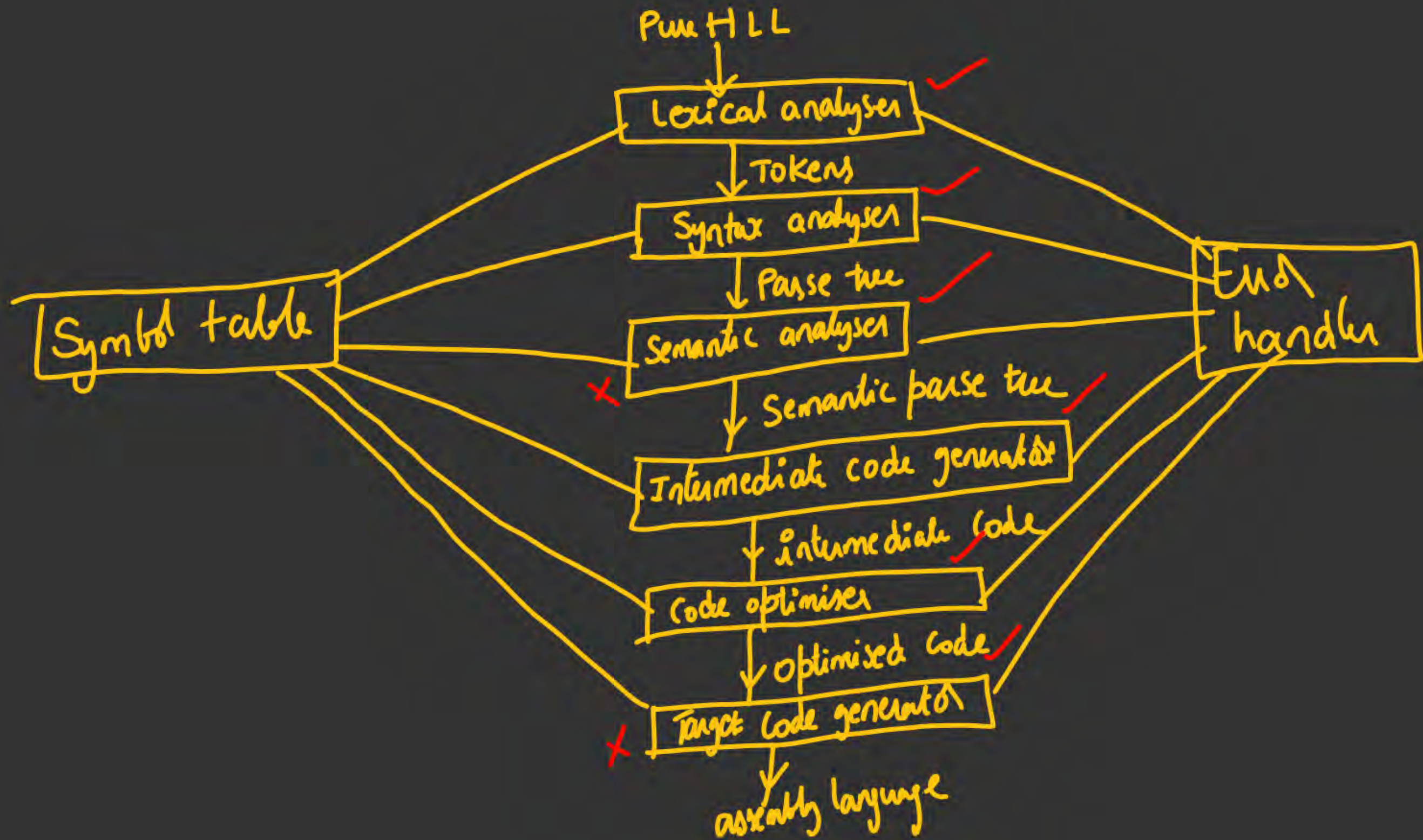
linker may take libraries such as OS libraries & user defined libraries & language libraries



loader:-

This is a module which loads an executable file into memory, initializes the registers, heaps, data etc and starts the execution of the program.

Address relocation is also done by loader.





## Design phases of Compiler :-

### The lexical analyzer :-

lexical analyzer is also called Scanner

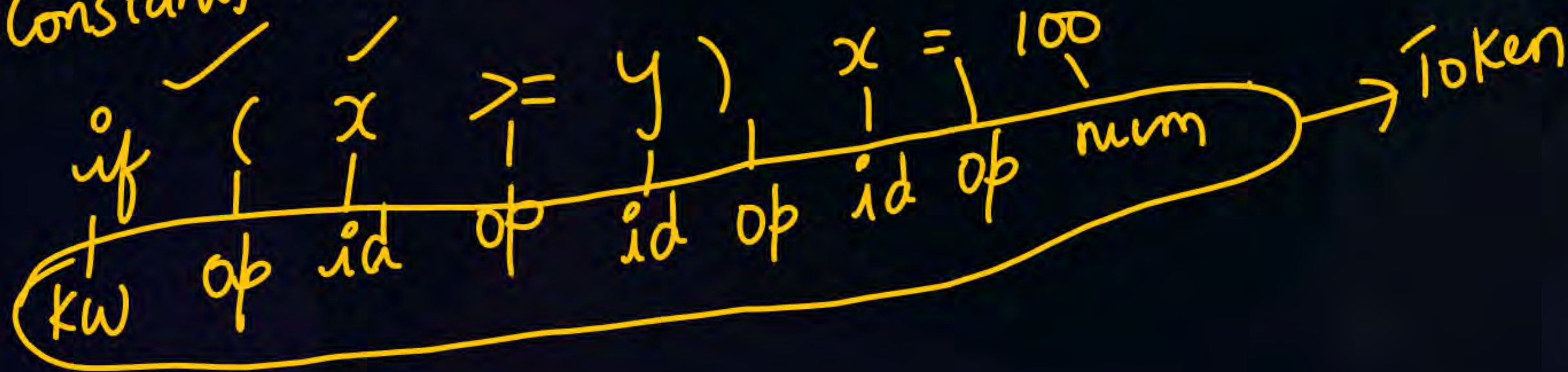
The Scanner groups the i/p stream of characters into a stream of tokens and constructs the

Symbol table, which will be used later



The tokens in any high level language can include

- Keywords
- identifiers
- operators
- Constants : numeric & character & strings





The i/p to lexical analyser is a character stream  
and o/p is stream of tokens.

Regular expressions are used to define the rules  
for recognising the tokens

lexical analyser is implemented as a Finite automata.

Automated Scanner generation tools are lex and flex

RE  $\rightarrow$  NFA  $\rightarrow$  DFA



**THANK - YOU**